

Machine Learning

12. Multi-Layer Perceptron (MLP)

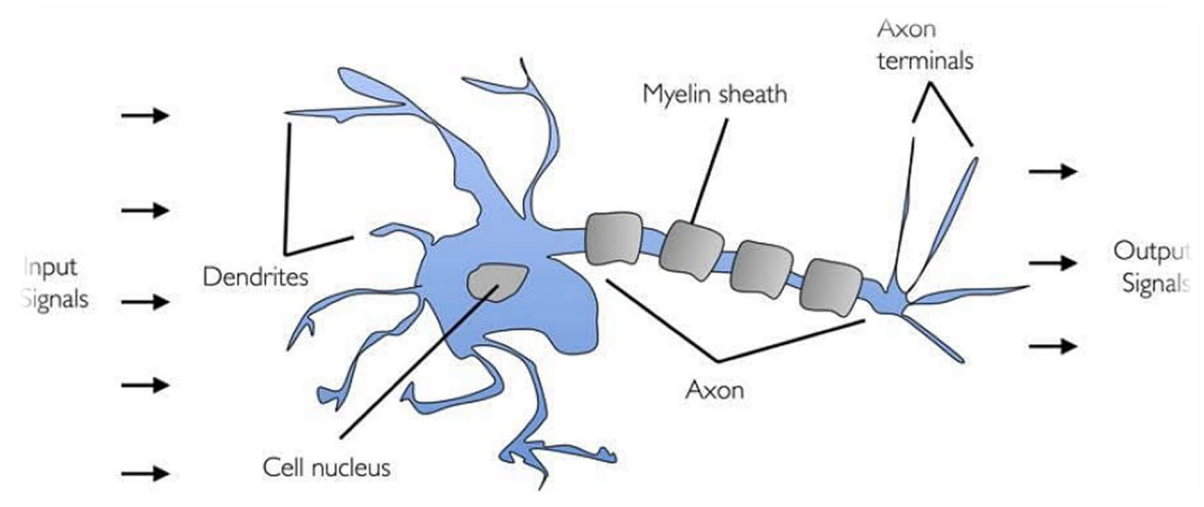
Sunwoo Lee
sunwool@inha.ac.kr



INHA UNIVERSITY

Today, we will learn...

- What is 'perceptron'?
- Why does it work?
- What can we do with it?



Do not share or distribute these slides!

저작권 안내 / Copyright Notice

본 사이트에서 수업 자료로 이용되는 저작물은 저작권법 제25조 수업목적저작물 이용 보상금제도의 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. **약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개·공유 및 수업 목적외의 사용을 금지합니다. (일부 공개로 제공되는 유튜브 강의를 불법으로 공유하거나 배포하는 행위를 금함)**

The teaching materials provided on this site are copyrighted and signed legally by the Korea Reproduction and Transmission Rights Association. Using these materials beyond the scope of the agreement is a violation against copyright laws. **Hence, one is not allowed to open or share the materials in public. Using them outside of the class is strictly prohibited.**

2020. 3. 11.

인하대학교·한국복제전송저작권협회

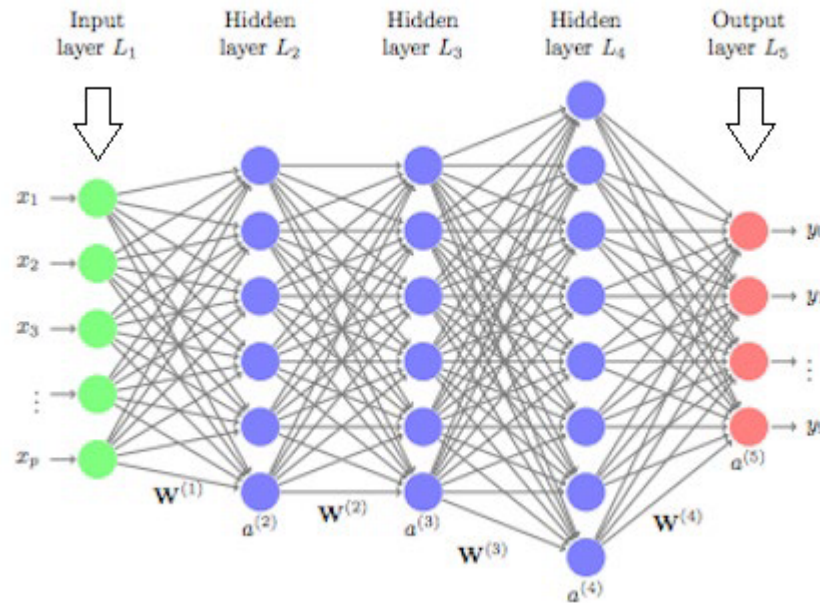
Inha University · Korea Reproduction and Transmission Rights Association

- ***A Brief History***

- Perceptron
- Multi-Layer Perceptron (MLP)
- Quick Introduction to Deep Neural Networks

Deep Learning?

- Artificial Neural Networks and its applications
- The larger the network, the better the learning capability!
- So, we focus on 'deep' networks.



A Brief History: Dawn of Deep Learning

- The Birth of Artificial Neural Networks

- McCulloch, Warren S. et al. proposed a novel concept of interpreting neural systems as a network of switches [1].
- This paper was published in 1943, and is considered as the very first article that proposed the concept of artificial neural networks.

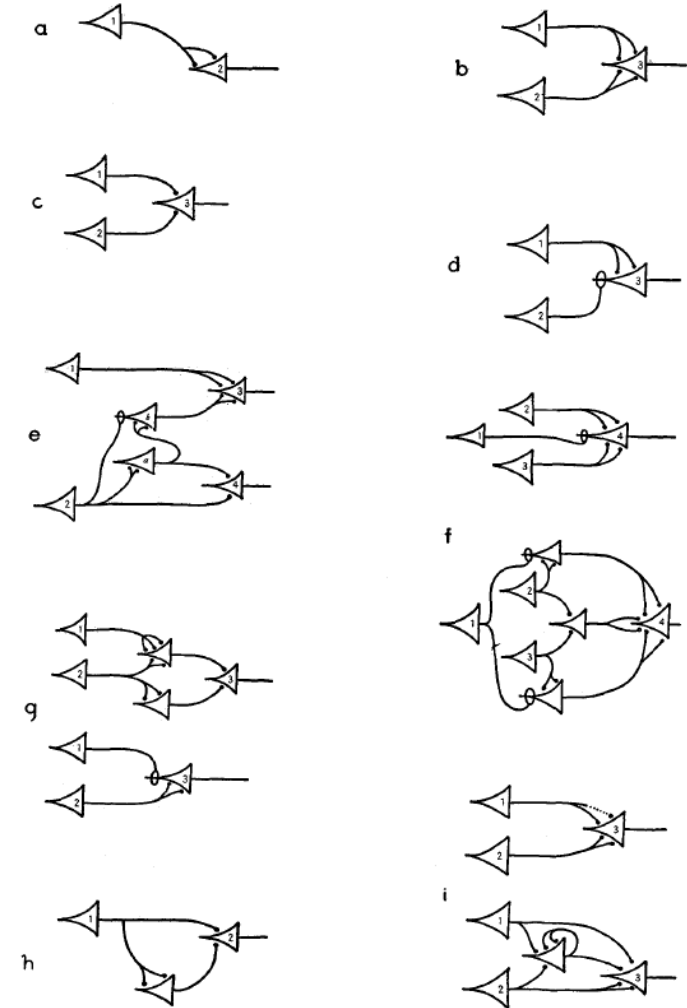
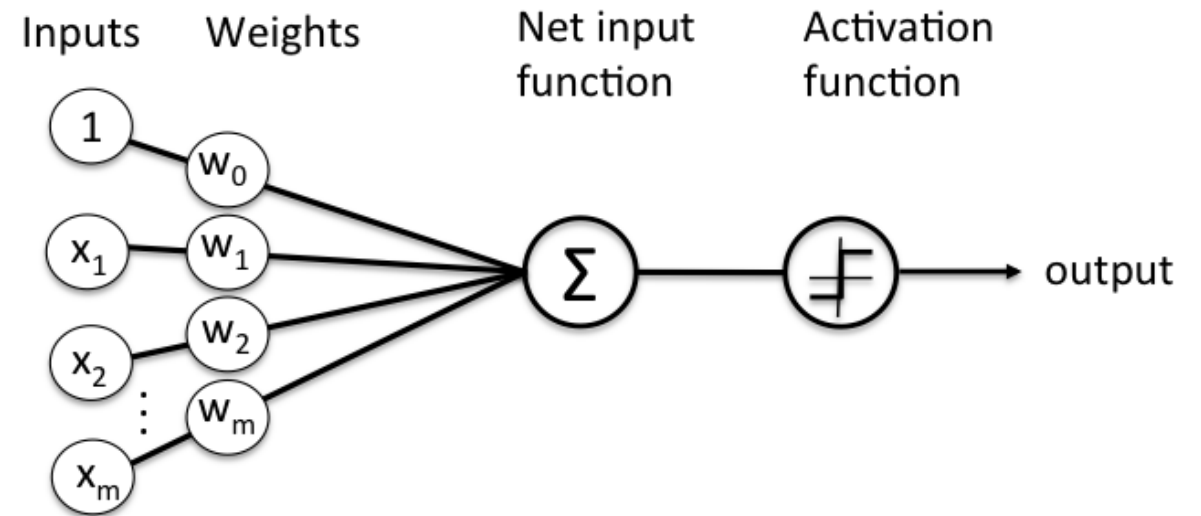


FIGURE 1

A Brief History: One Big Step Forward!

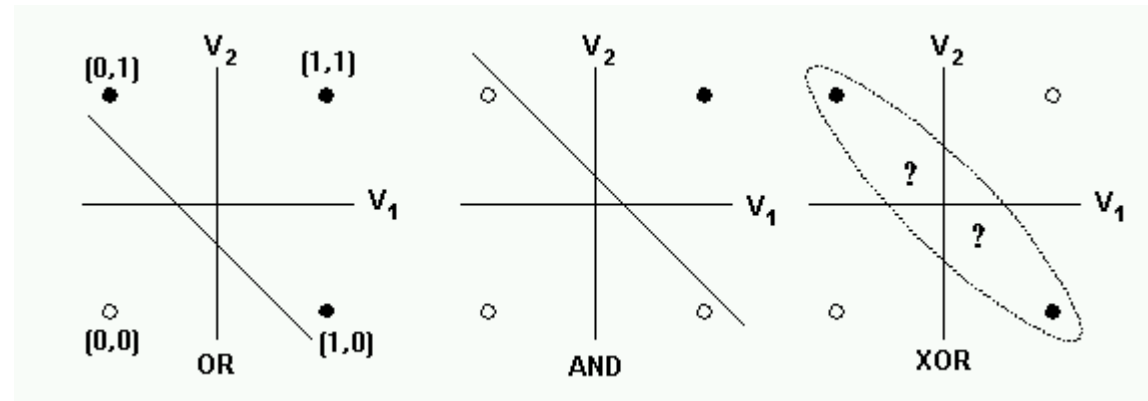
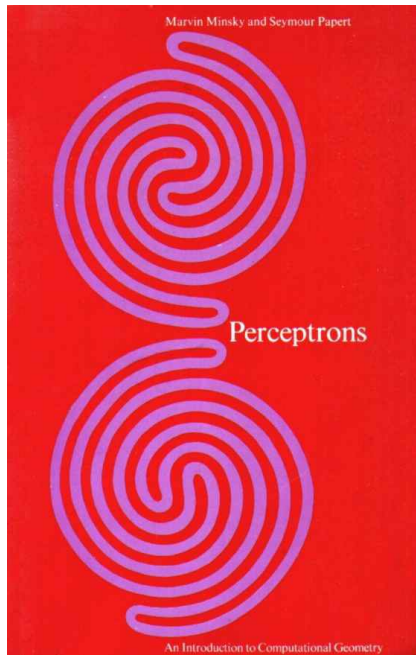
- Perceptron
 - A linear transformation mechanism.
 - Proposed by Frank Rosenblatt et al., in 1957, The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain.



Schematic of Rosenblatt's perceptron.

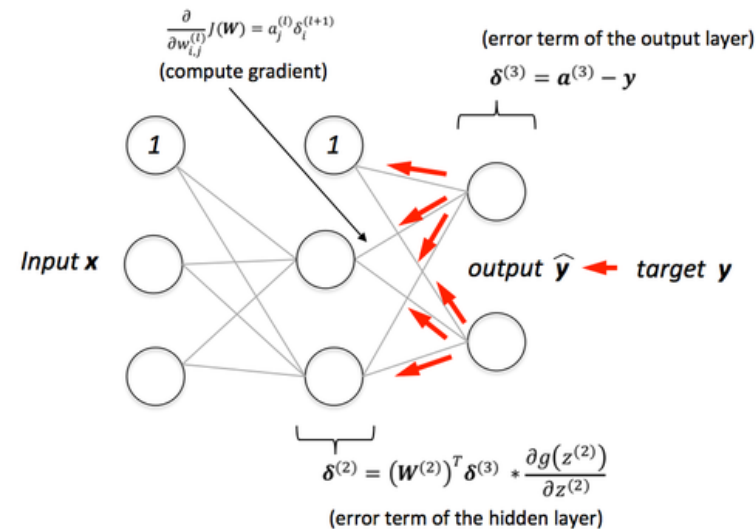
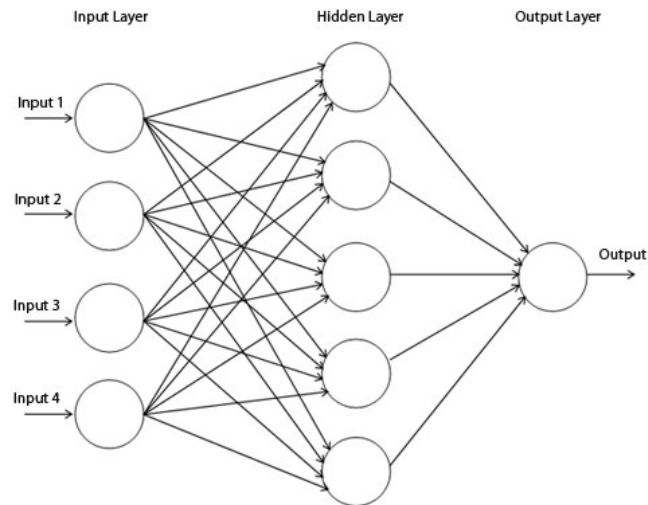
A Brief History: However...

- In 1969, Minsky et al. mathematically proved the limitations of perceptron.
- Then, the researchers quickly lost interests in the perceptron.



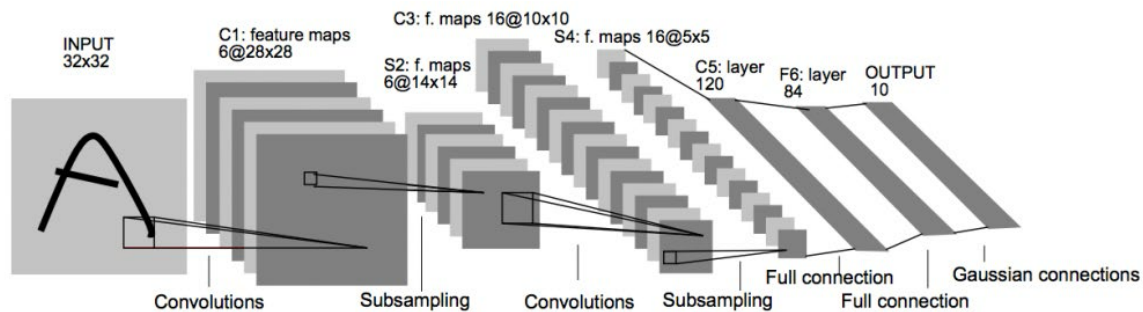
A Brief History: Resurrection

- In 1986, Geoffrey Hinton et al. wrote 'Parallel Distributed Processing' and introduced 'Multi-Layer Perceptron' (MLP).
- To train MLP, the authors also proposed 'backpropagation'.

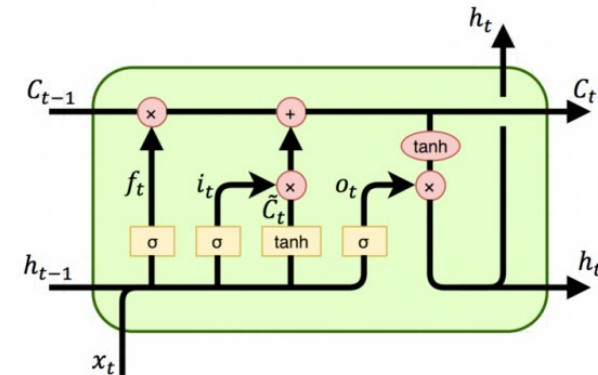


A Brief History: Bloom Again

- Artificial Neural Networks have been proved to be useful in many applications.
 - In 1998, Yann Lecun proposed a very first Convolutional Neural Network that classifies images very well.
 - In 1997, Long-Short Term Memory (LSTM) has been proposed by Sepp Hochreiter et al.



CNN for computer vision tasks



LSTM for language modeling tasks

A Brief History: Second Winter?

- However, deep networks suffer from 'gradient vanishing' problems (known for the first time in 2000).
- Training takes too much resources (time, compute power, etc).
- Strong competitors appeared (SVM with kernel).



A Brief History: Keep Going On Anyway!

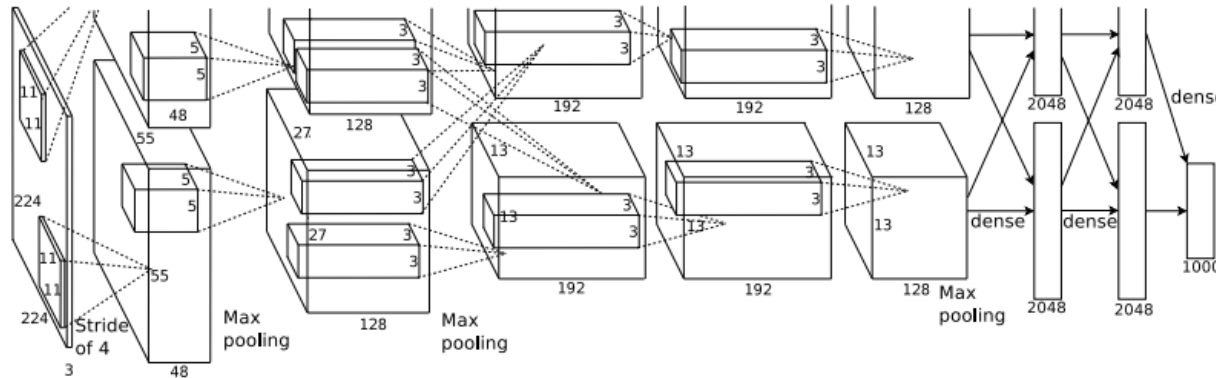
- Many techniques effectively solved the gradient vanishing problem (such as Batch Normalization).
- Advanced types of neural network demonstrated their efficacy in many applications.
 - Speech Recognition
 - Computer Vision
 - Pattern Recognition
 - Etc.



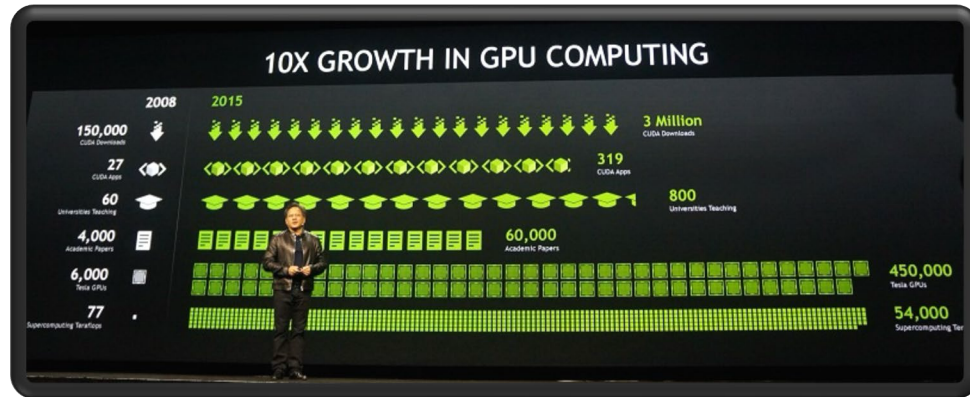
The name, 'deep learning' has gained the popularity from mid-2000.

A Brief History: Breakthrough

- In 2012, Alex shocked the world in ImageNet Classification Competition.
 - ‘AlexNet’, a deep convolutional neural network designed by Alex Krizhevsky.
 - AlexNet pushed top-5 error from 26.1% to 15.3% at once.

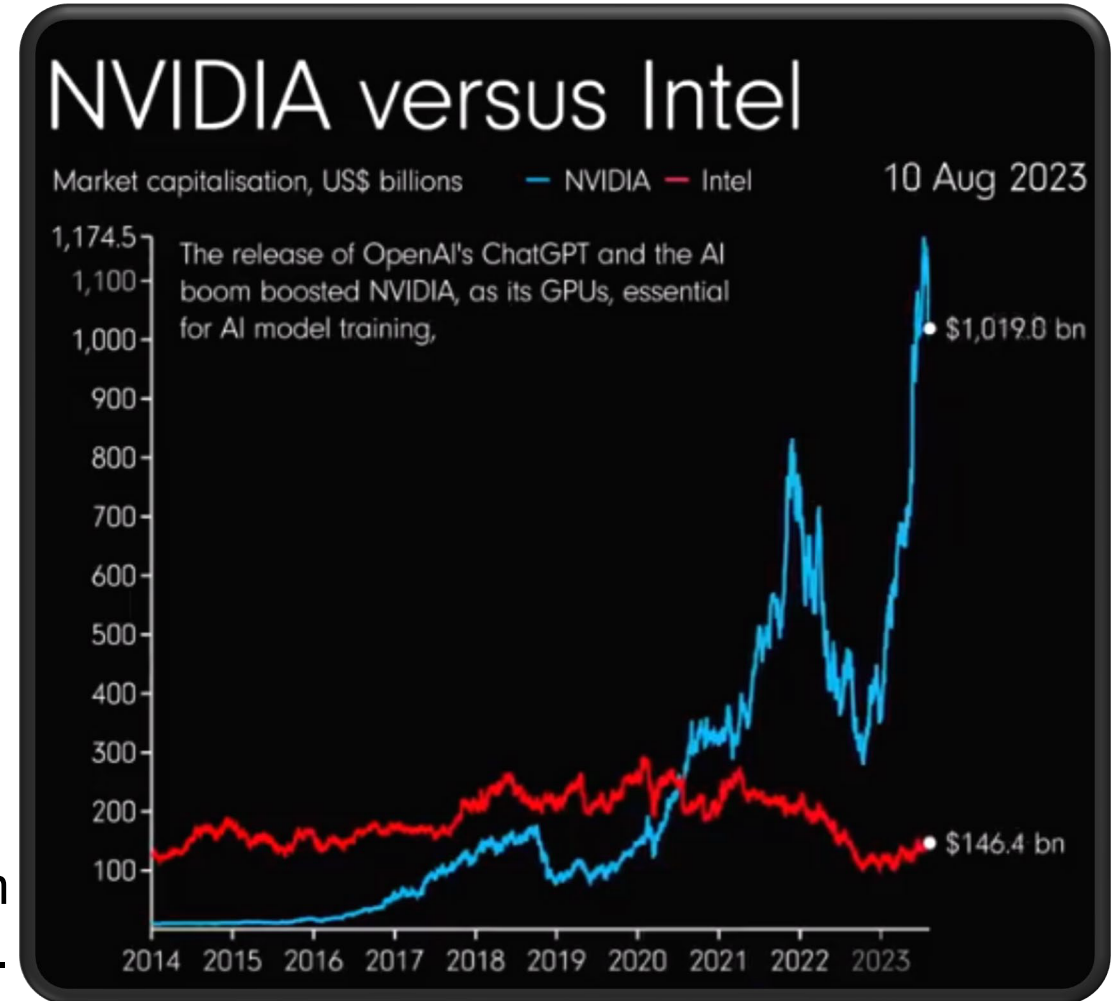


A Brief History: Advances in Hardware



Jen-Hsun Huang, CEO of Nvidia in GTC 2015

GPU has become the most dominant system resource for data analysis.



A Brief History: Full Bloom

- Many real-world applications start to take advantage of deep learning.
- Industry dived into the AI field (Google, Facebook, Amazon, Microsoft, etc.).



Google's AlphaGo



Meta's DeepFace

A Brief History: Now?

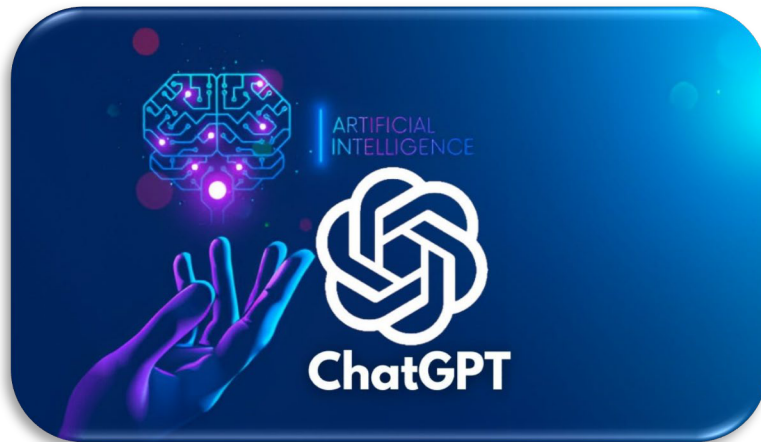
- Nowadays, DL is considered as modern ML.
- Artificial Neural Network is usually considered as a ***black-box*** algorithm
- Many software packages for deep learning
 - TensorFlow, JAX, PyTorch, Caffe2, MxNet, etc.
- However, a variety of theoretical and practical techniques are hidden behind the black-box algorithm (neural networks).



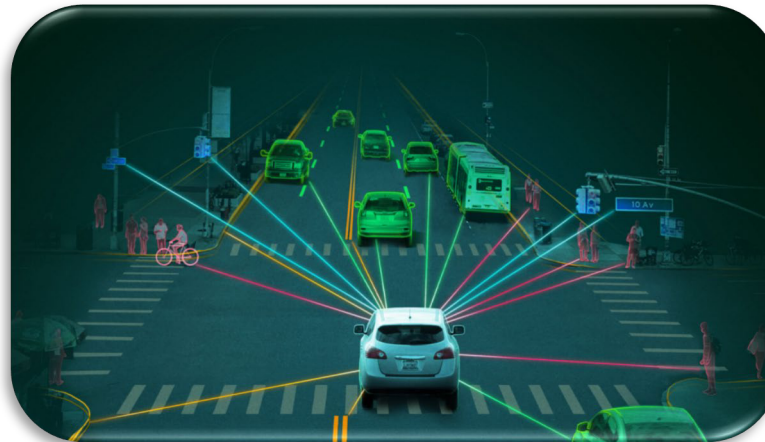
TensorFlow

P Y T  R C H

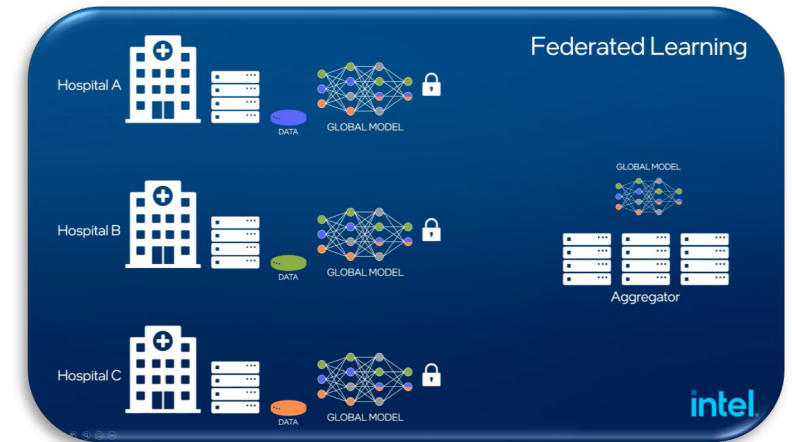
Deep Learning is Changing the World!



Large Language Model
(LLM)



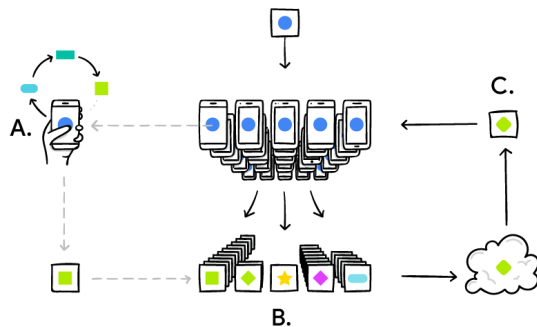
Object Detection for
Self-Driving



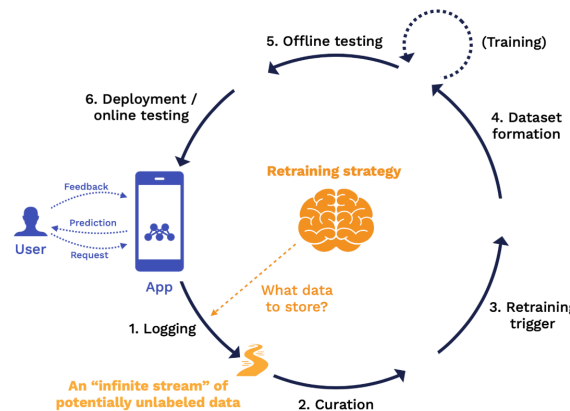
Distributed Learning for
Medical Data Analysis

A Brief History: On-Going Evolution

- Many interesting research works are on-going!
 - Federated Learning
 - Continual Learning
 - Generative Learning
 - Meta Learning
 - Large-Scale Language Modeling (LLM)



Federated Learning



Continual Learning



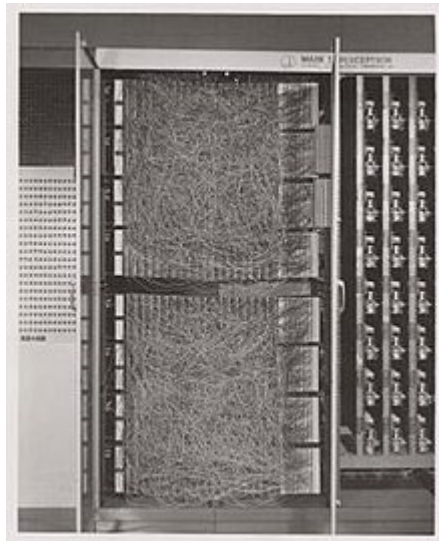
LLM-based IoT Devices

- A Brief History
- ***Perceptron***
- Multi-Layer Perceptron (MLP)
- Quick Introduction to Deep Neural Networks

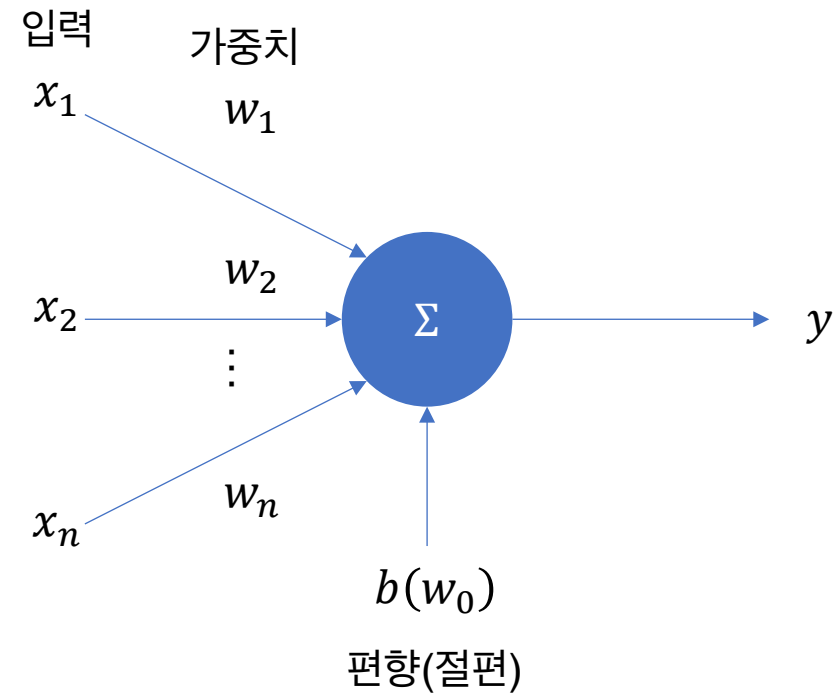
Perceptron

- A simple algorithm for binary classification tasks.

Y를 그대로 사용하면 선형회귀와 동일

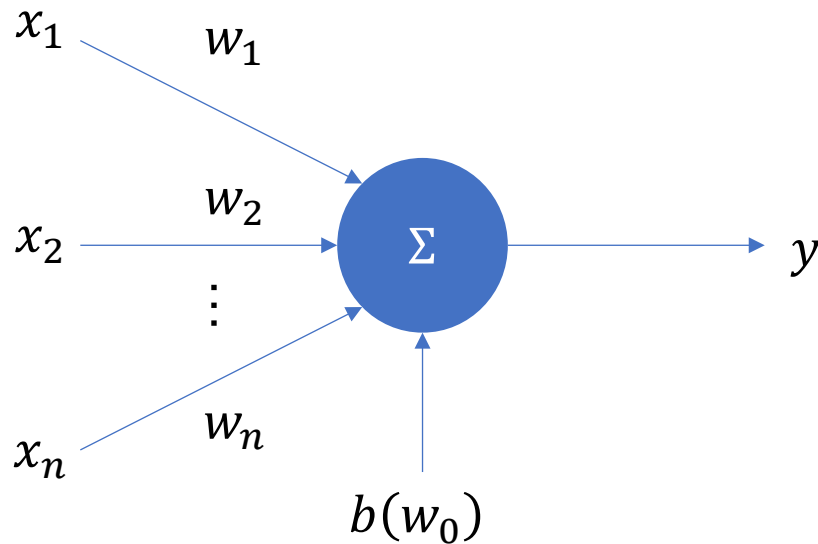


Mark I Perceptron machine,
the first implementation of
the perceptron algorithm.



Perceptron Mechanism

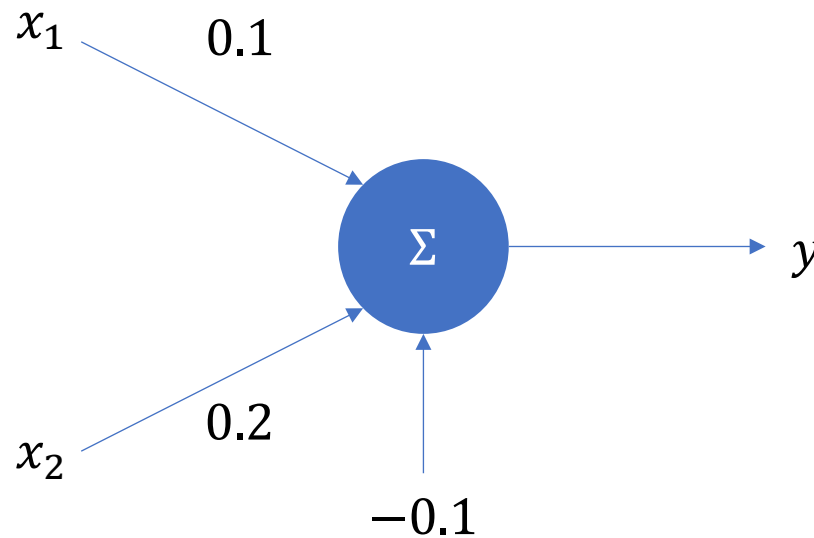
- Multiple inputs and a single binary output.
- The bias b works as a threshold.



$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0, \\ 0 & \text{otherwise} \end{cases}$$

Example: Perceptron

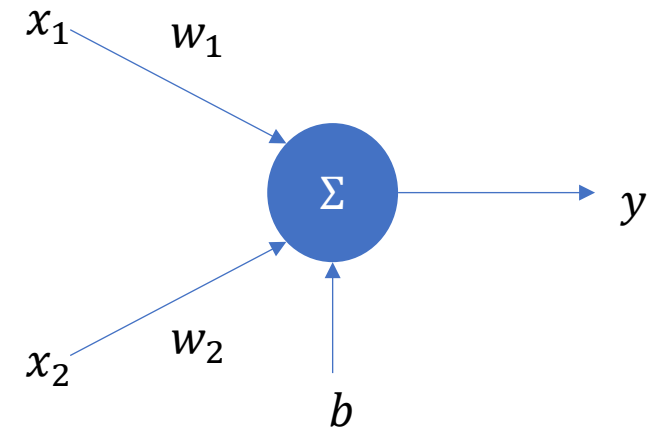
- For given $x = (0.4, 0.7)$, the perceptron yields 0.08.
- Because the output is > 0 , $y = 1$!



Versatile Perceptron

- AND Gate

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

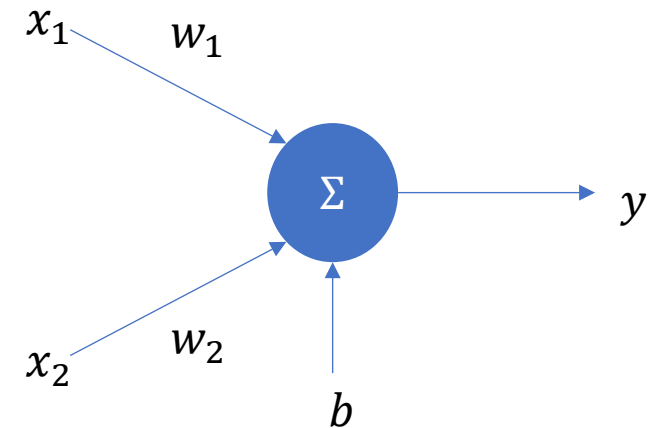


- Which one is the solution (w_1 , w_2 , and b)?
 - (0.4, 0.5, -0.3)
 - (0.5, 0.5, -0.6)
 - (0.6, 0.6, -0.5)

Versatile Perceptron

- OR Gate

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

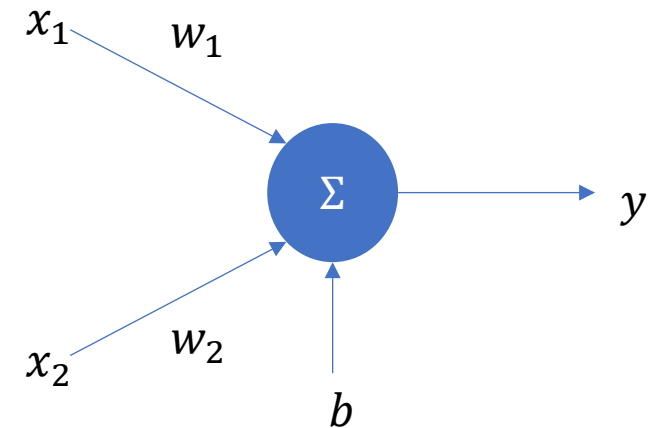


- Which one is the solution (w_1 , w_2 , and b)?
 - (0.4, 0.5, -0.3)
 - (0.5, 0.5, -0.6)
 - (0.6, 0.1, -0.5)

Versatile Perceptron

- NAND Gate

x_1	x_2	y
0	0	1
1	0	1
0	1	1
1	1	0

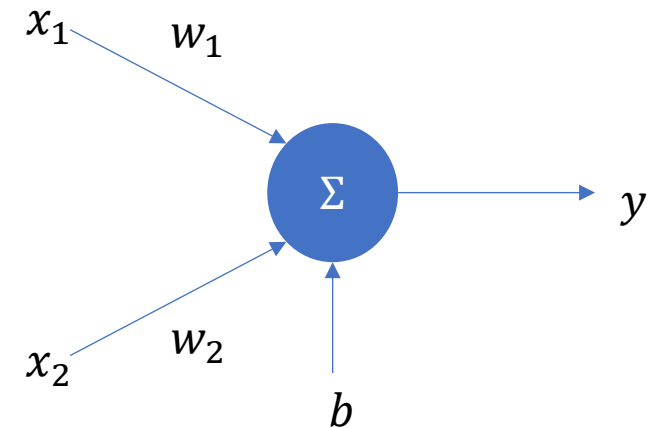


- Which one is the solution (w_1 , w_2 , and b)?
 - $(-0.1, -0.5, 0.6)$
 - $(0.5, 0.5, -0.1)$
 - $(0.6, -0.1, 0.5)$

Versatile Perceptron

- XOR Gate

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0



- Which one is the solution (w_1 , w_2 , and b)?
 - $(-0.1, -0.5, 0.6)$
 - $(0.5, 0.5, -0.1)$
 - $(0.6, -0.1, 0.5)$

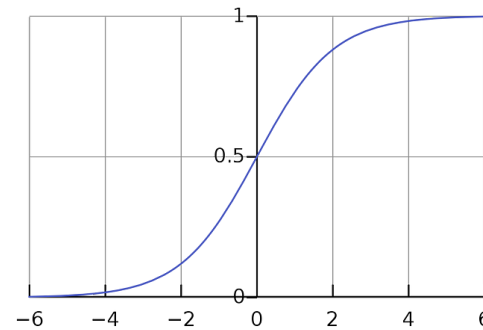
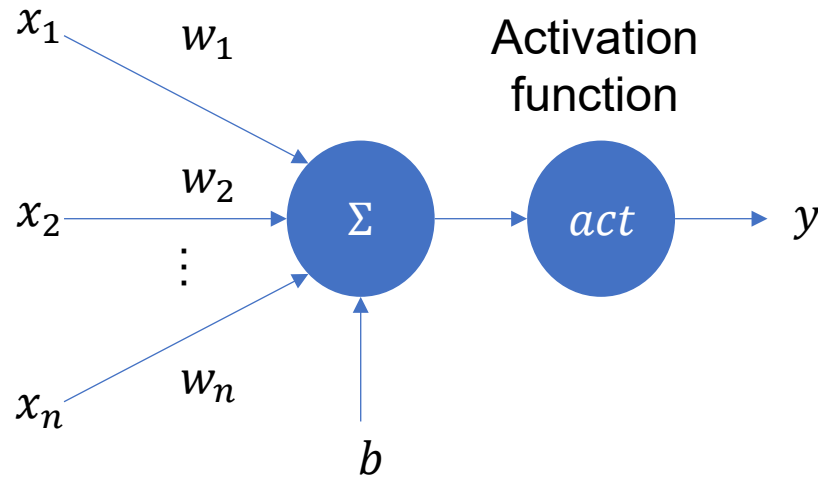
퍼셉트론은 비선형 패턴만 학습 가능

No solution! This is the limitation of perceptron!

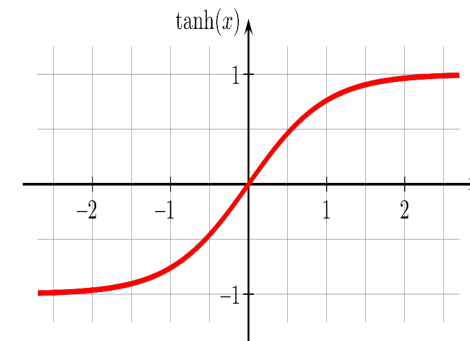
Activation Function

출력의 범위를 제어

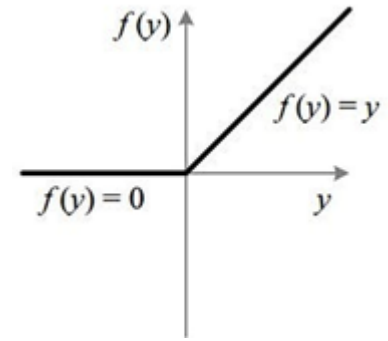
- Attached to the end of perceptrons to control the output range.
 - Sigmoid (probability output) = 로지스틱 회귀
 - Hyperbolic tangent (binary classification)
 - ReLU (... just works well...)



Sigmoid



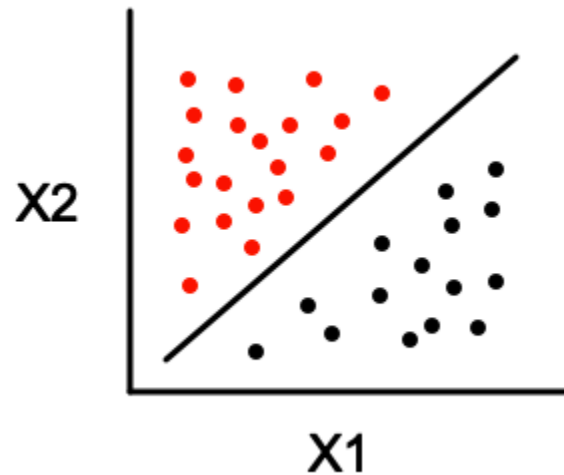
Tanh



ReLU

Perceptron As A Classifier

- Perceptron with `tanh` activation: binary classification
- Perceptron with `sigmoid` activation: logistic regression



Training Perceptron (1/5)

- The same as other ML algorithms.
 - Define a loss function.
 - Solve it using any gradient-based optimization algorithm.

- MSE

$$L(w) = \frac{1}{2} \sum_{i=1}^n (f(y) - \bar{y})^2$$

$$\frac{\partial L(w)}{\partial w} = \frac{\partial L(w)}{\partial f(y)} \cdot \frac{\partial f(y)}{\partial y} \cdot \frac{\partial y}{\partial w}$$

$$\frac{\partial L(w)}{\partial b} = \frac{\partial L(w)}{\partial f(y)} \cdot \frac{\partial f(y)}{\partial y} \cdot \frac{\partial y}{\partial b}$$

Chain-rule

출력 Y를 활성화 함수에 입력한 결과

Gradient Descent

$$w = w - \eta \frac{\partial L(w)}{\partial w}$$

$$b = b - \eta \frac{\partial L(w)}{\partial b}$$

Training Perceptron (2/5)

- Derivatives of activation functions

$$\bullet \frac{\partial L(w)}{\partial w} = \frac{\partial L(w)}{\partial f(y)} \cdot \frac{\partial f(y)}{\partial y} \cdot \frac{\partial y}{\partial w}$$

$$L(w) = \frac{1}{2} (f(y) - \bar{y})^2$$



$$\frac{\partial L(w)}{\partial f(y)} = (f(y) - \bar{y})$$

Training Perceptron (3/5)

- Derivatives of activation functions 활성화 함수는 미분 가능해야함

- $$\frac{\partial L(w)}{\partial w} = \frac{\partial L(w)}{\partial f(y)} \cdot \frac{\partial f(y)}{\partial y} \cdot \frac{\partial y}{\partial w}$$

- Sigmoid:
$$\begin{aligned}\sigma'(y) &= \frac{\partial}{\partial y} \frac{1}{1+e^{-y}} = \frac{\partial}{\partial y} (1 + e^{-y})^{-1} = -(1 + e^{-y})^{-2} \frac{\partial}{\partial y} 1 + e^{-y} \\ &= -(1 + e^{-y})^{-2} \cdot -e^{-y} = \frac{e^{-y}}{(1+e^{-y})^2} = \frac{1}{1+e^{-y}} \cdot \frac{e^{-y}}{1+e^{-y}} = \sigma(y)(1 - \sigma(y))\end{aligned}$$

- ReLU:
$$\sigma(y) = \begin{cases} y & \text{If } y > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\sigma'(y) = \begin{cases} 1 & \text{If } y > 0 \\ 0 & \text{otherwise} \end{cases}$$

Note that ReLU is not differentiable at a point of (0, 0).

(0, 0)에서 미분 불가능, 모델 성능을 '어떻게' 높이는지를 설명

할 수 없음, 다만 입력이 정확히 0만 아니면 문제가 되지 않음

Training Perceptron (4/5)

- Derivatives of activation functions

- $\frac{\partial L(w)}{\partial w} = \frac{\partial L(w)}{\partial f(y)} \cdot \frac{\partial f(y)}{\partial y} \cdot \frac{\partial y}{\partial w}$

$$y = w \cdot x + b$$



$$\frac{\partial y}{\partial w} = x$$

Training Perceptron (5/5)

- Derivatives of activation functions

- $$\frac{\partial L(w)}{\partial w} = \frac{\partial L(w)}{\partial f(y)} \cdot \frac{\partial f(y)}{\partial y} \cdot \frac{\partial y}{\partial w}$$

$$= \underbrace{(f(y) - \bar{y})}_{\text{error}} \cdot \underbrace{(\sigma(y)(1 - \sigma(y)))}_{\text{sigmoid error}} \cdot \underbrace{x}_{\text{input}}$$

error

sigmoid error

input

퍼셉트론의 출력에 대한 오류

오류를 유발한 입력

입력에 대한 활성화 함수의 오류 기여도

The gradient is composed of not only how much it is inaccurate but also which information makes such an error!

Perceptron is Simple and Powerful, However...

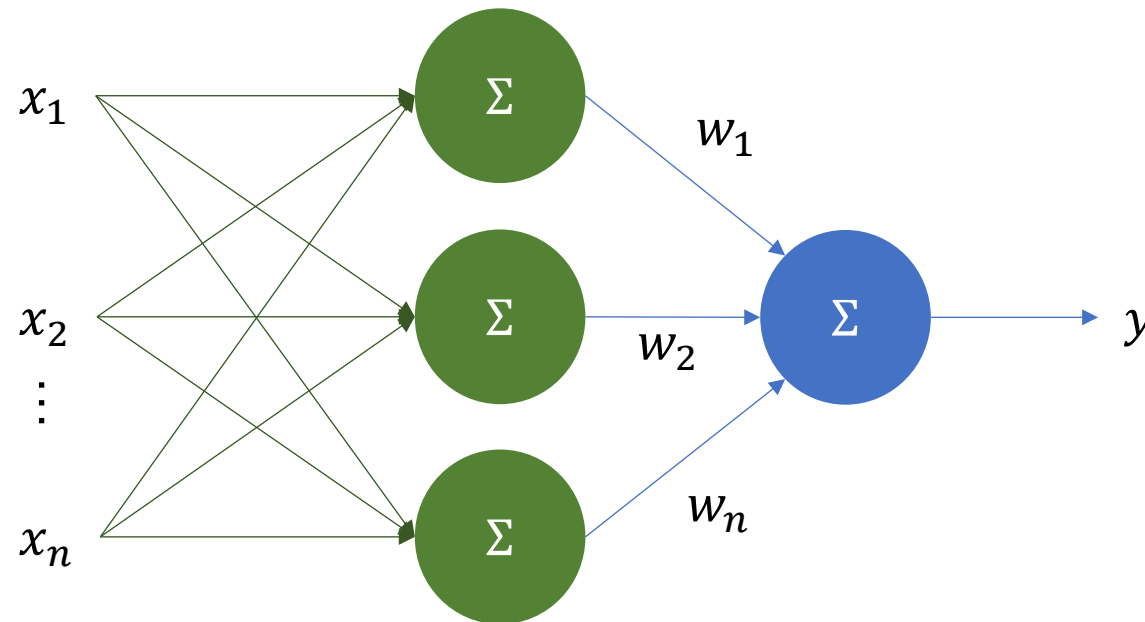
- Has an apparent limitation of the representational limits.
 - The output is only a binary.
 - It only works for the linearly-separable data.
- After the AI winter in 1980s~1990s, perceptron resurrected as an even more powerful algorithm without any representational limits!



- A Brief History
- Perceptron
- ***Multi-Layer Perceptron (MLP)***
 - ***Concept***
 - Training (Backpropagation)
- Quick Introduction to Deep Neural Networks

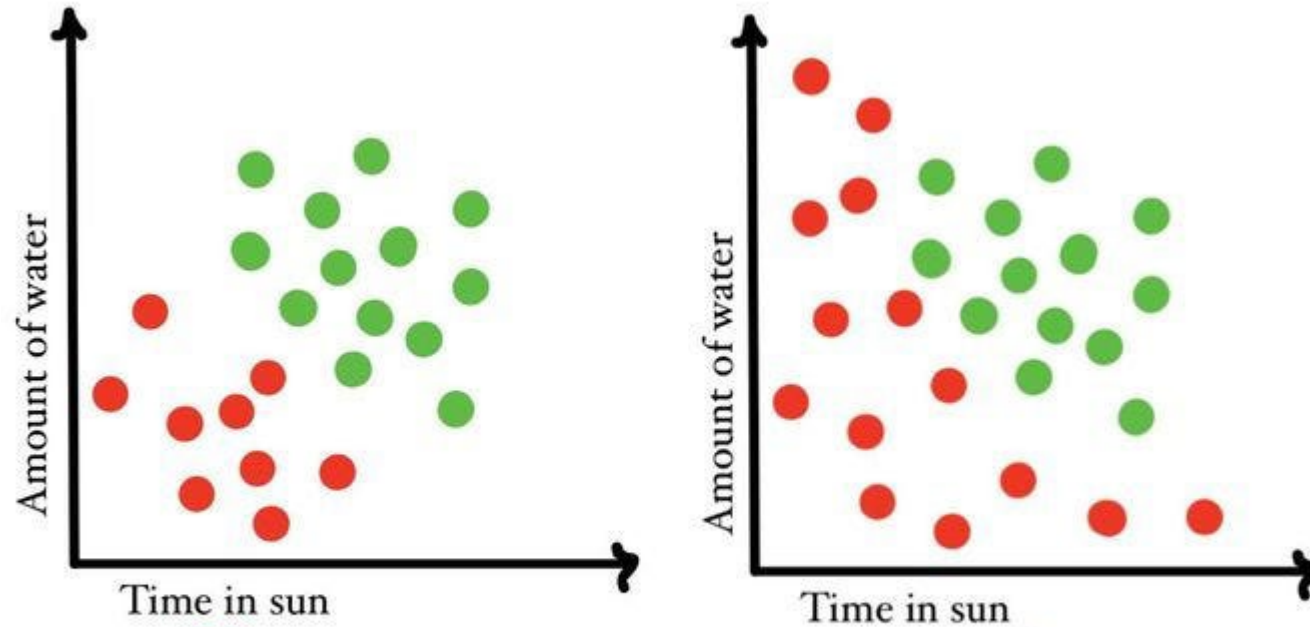
Multi-Layer Perceptron

- A set of layers each of which consists of multiple perceptron.
 - The term 'MLP' is used ambiguously, sometimes loosely to mean any feedforward artificial neural networks. 수학으로 비유하면 합성함수의 매커니즘과 동일



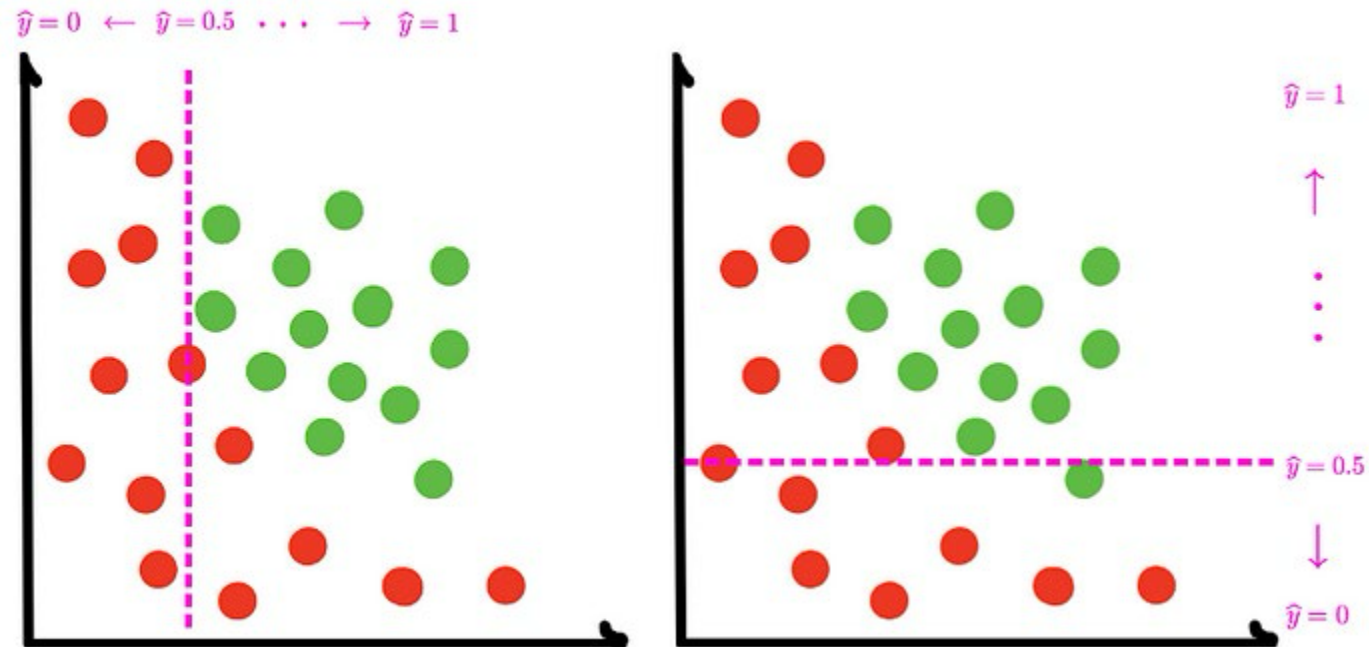
Why Is It Better than Perceptron? (1/3)

- The most significant drawback of perceptron: it only works for linearly-separable data.



Why Is It Better than Perceptron? (2/3)

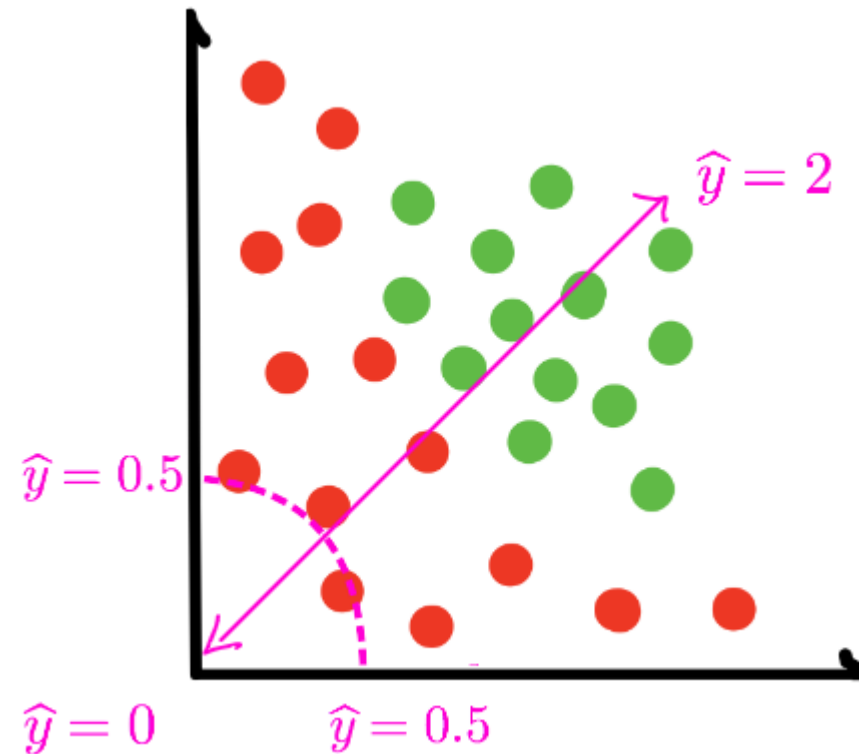
- Multi-Layer Perceptron can be considered as a summation of multiple linear models! Y축과 X축을 나누어서 해석



Can we separate these data points with two linear models?

Why Is It Better than Perceptron? (3/3)

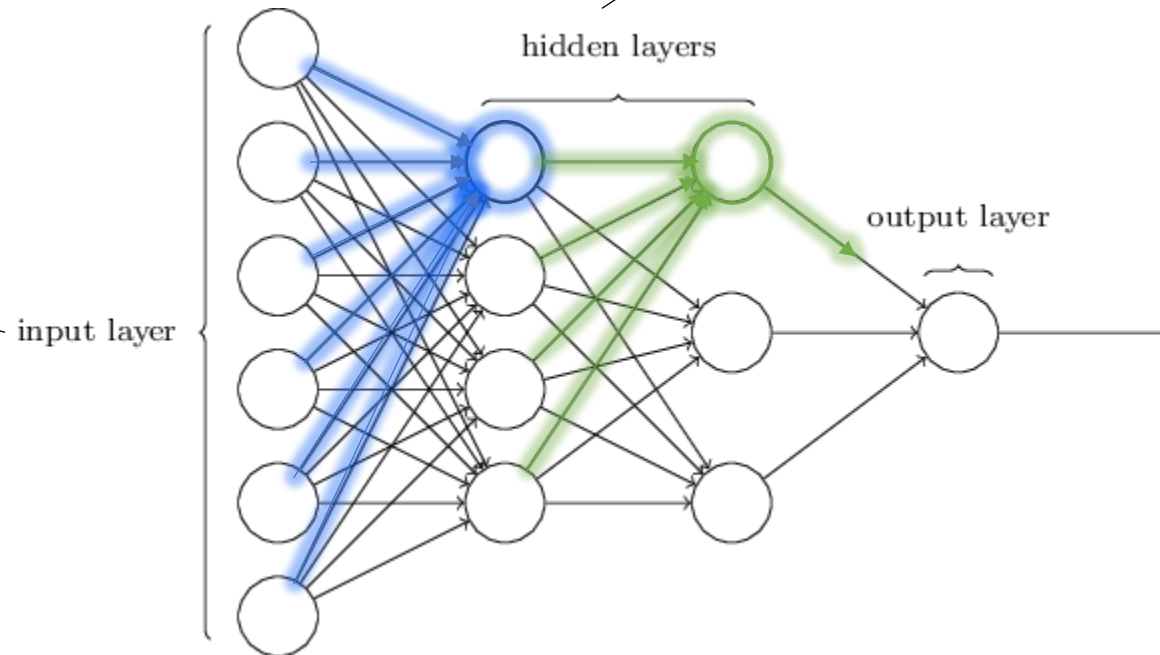
- A linear combination of multiple linear models can lead to a model with a non-linear decision boundary. 거리와 방향을 나누어서 해석



Structure

Having these layers is the breakthrough idea!

Just the input data points



Natural Questions Here.

Q1: How many layers should my model have?

- There is no universal rule. However, it is known that, the more the layers, the stronger the learning capability.

Q2: How many neurons should each layer have?

- It depends on the problem!
- The hidden layers: Likely to the number of layers, the larger the layer, the stronger the learning capability.

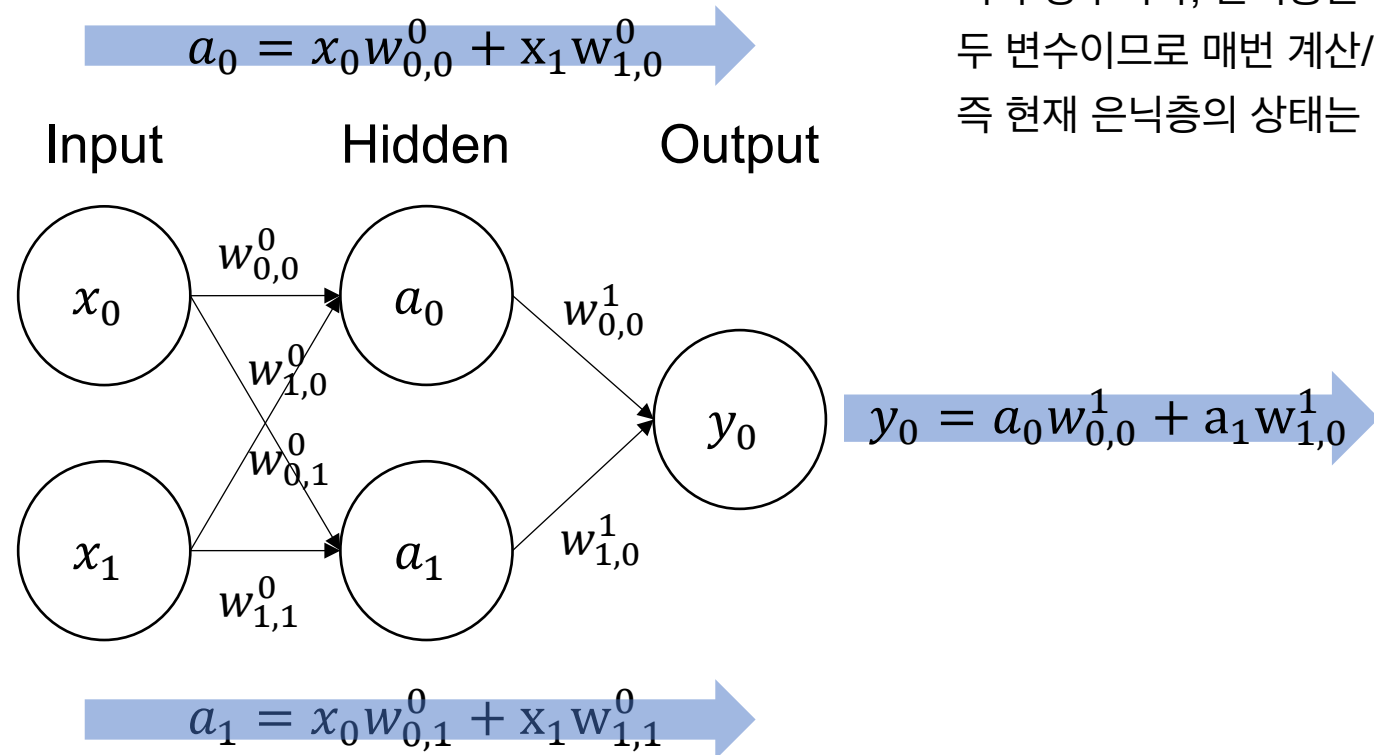
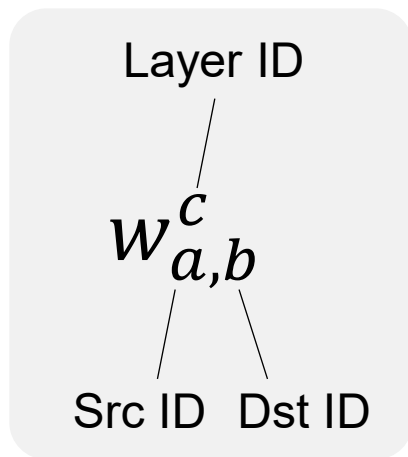
기본적으로 답이 없으므로 하이퍼파라미터로 설정하여 튜닝해야함, 심지어 문제마다 다름

- A Brief History
- Perceptron
- ***Multi-Layer Perceptron (MLP)***
 - Concept
 - ***Training (Backpropagation)***
- Quick Introduction to Deep Neural Networks

Data Dependency (1/2)

- MLP has a special feature, data dependency.

입력층과 출력층은 입출력 중 한 쪽 레이어가 상수이나, 은닉층은 입출력 양쪽 모두 변수이므로 매번 계산/갱신마다 변함, 즉 현재 은닉층의 상태는 의미가 없음



데이터 의존성 발생, 선행 레이어의 계산이 끝나야 후행 레이어의 계산이 가능 = 계산의 방향성이 생김 = 추론(Feed Forward)

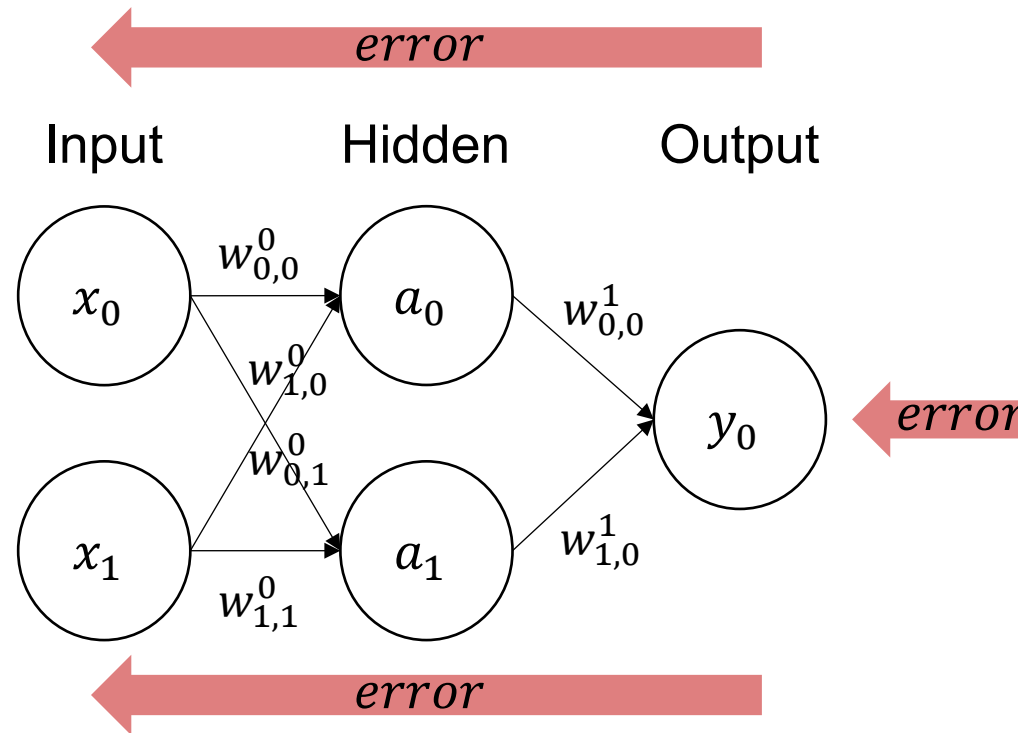
Data Dependency (2/2)

- Due to the forward direction data dependency, finding a good value of the hidden layer parameters is extremely difficult.
 - The best parameter values at the hidden layer depends on the input data that constantly changes throughout the training!
- As a solution, *backpropagation* has been proposed by several researchers in a similar period of time (1980~1990)

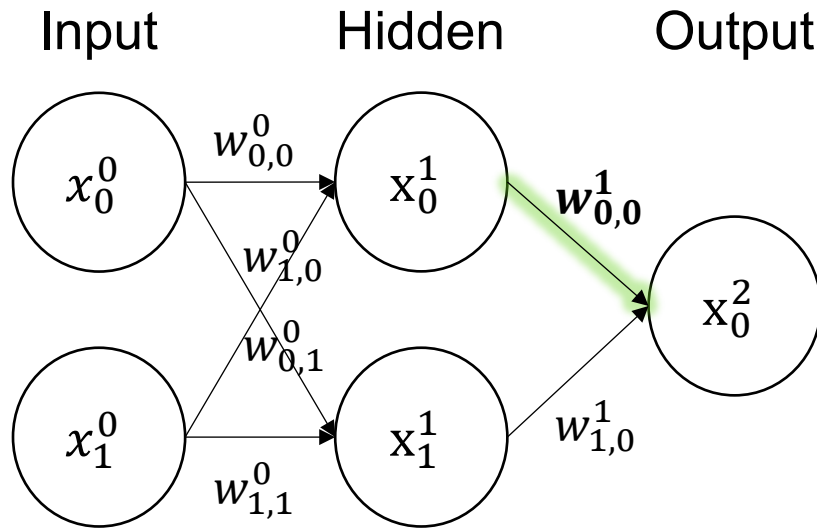
Backpropagation

- An algorithm that calculates the gradients, going through the network layers in a backward direction.

최종 출력값의 오류를 바탕으로 출력층의 선행 레이어 각각에 대한 오류를 거슬러 가면서 계산 가능 = 데이터 의존성이 역방향으로 발생



Backpropagation at the Output Layer



- $L(w) = \frac{1}{2} (x_0^2 - \bar{y})^2$
- $x_0^2 = f(z_0^2)$
- $z_0^2 = x_0^1 w_{0,0}^1 + x_1^1 w_{1,0}^1$
- $\frac{\partial L(w)}{\partial w_{0,0}^1} = \underbrace{\frac{\partial L(w)}{\partial x_0^2}}_{\textcircled{1}} \cdot \underbrace{\frac{\partial x_0^2}{\partial z_0^2}}_{\textcircled{2}} \cdot \underbrace{\frac{\partial z_0^2}{\partial w_{0,0}^1}}_{\textcircled{3}}$

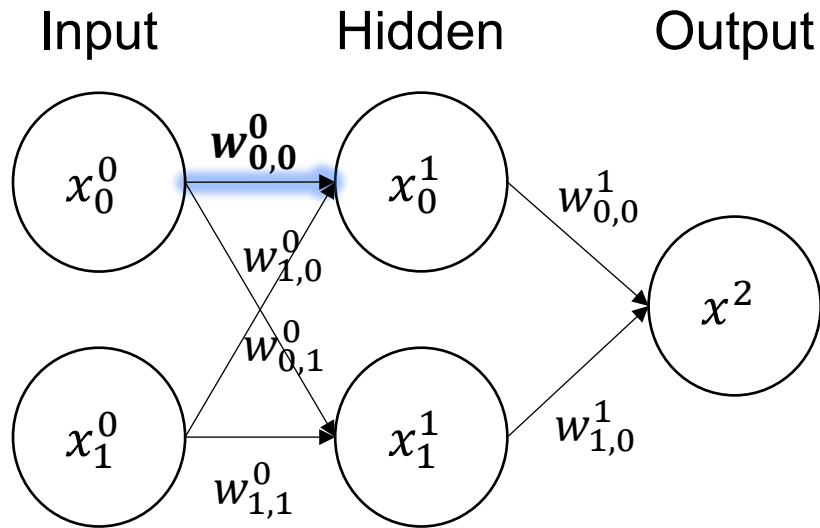
$$\textcircled{1} \frac{\partial L(w)}{\partial x_0^2} = \left(\frac{1}{2} (x_0^2 - \bar{y})^2 \right)' = x_0^2 - \bar{y}$$

$$\textcircled{2} \frac{\partial x_0^2}{\partial z_0^2} = f'(z_0^2) = \sigma(z_0^2) (1 - \sigma(z_0^2)) \quad \textcircled{3} \frac{\partial z_0^2}{\partial w_{0,0}^1} = x_0^1$$



$$\frac{\partial L(w)}{\partial w_{0,0}^1} = (x_0^2 - \bar{y}) \sigma(z_0^2) (1 - \sigma(z_0^2)) x_0^1$$

Backpropagation at the Hidden Layer



- $L(w) = \frac{1}{2} (x_0^2 - \bar{y})^2$

- $x_0^1 = f(z_0^1)$

- $z_0^1 = x_0^0 w_{0,0}^0 + x_1^0 w_{1,0}^0$

- $$\frac{\partial L(w)}{\partial w_{0,0}^0} = \underbrace{\frac{\partial L(w)}{\partial z_0^1}}_{\textcircled{1}} \cdot \underbrace{\frac{\partial z_0^1}{\partial w_{0,0}^0}}_{\textcircled{2}} = \underbrace{\frac{\partial L(w)}{\partial z_0^2}}_{\textcircled{1}} \cdot \underbrace{\frac{\partial z_0^2}{\partial z_0^1}}_{\textcircled{2}} \cdot \underbrace{\frac{\partial z_0^1}{\partial w_{0,0}^0}}_{\textcircled{3}}$$

후행 레어에 대한 데이터 의존성 발생

① $\frac{\partial L(w)}{\partial z_0^2} =$ We already have it in the previous step!

② $\frac{\partial z_0^2}{\partial z_0^1} = \frac{\partial x_0^1 w_{0,0}^1 + x_1^1 w_{1,0}^1}{\partial z_0^1} = \frac{\partial f(z_0^1) w_{0,0}^1 + f(z_1^1) w_{1,0}^1}{\partial z_0^1} = f'(z_0^1) w_{0,0}^1 = \sigma(z_0^1) (1 - \sigma(z_0^1)) w_{0,0}^1$

③ $\frac{\partial z_0^1}{\partial w_{0,0}^0} = x_0^0$



$$\frac{\partial L(w)}{\partial w_{0,0}^0} = \frac{\partial L(w)}{\partial z_0^2} \sigma(z_0^1) (1 - \sigma(z_0^1)) w_{0,0}^1 x_0^0$$

Backpropagation Summary

- We have calculated two sets of weight gradient:

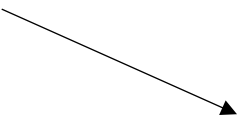
$$\begin{aligned} \bullet \frac{\partial L(w)}{\partial w_{0,0}^1} &= x_0^2 - \bar{y} \cdot \sigma(z_0^2) (1 - \sigma(z_0^2)) \cdot x_0^1 \\ \bullet \frac{\partial L(w)}{\partial w_{0,0}^0} &= \frac{\partial L(w)}{\partial z_0^2} \cdot w_{0,0}^1 \sigma(z_0^1) (1 - \sigma(z_0^1)) \cdot x_0^0 \end{aligned}$$

The diagram illustrates the backpropagation of error. Two equations are shown. The first equation's right-hand side terms are circled in orange. The second equation's right-hand side terms are also circled in orange. An orange arrow points from the $\frac{\partial L(w)}{\partial z_0^2}$ term in the second equation to the $\sigma(z_0^2)$ term in the first equation, indicating the flow of error from the second layer back to the first layer. To the right of the equations, there are two blue circles labeled x_0^1 and x_0^0 . Blue arrows point from these circles to the text "Input data".

- In the backpropagation, the errors are propagated backward, and so do the gradients.

Backpropagation is A General Algorithm

- The algorithm works perfectly well for any neural networks:
 - Regardless of the number of layers
 - Regardless of the size of each layer
 - Regardless of the pattern of the connections
- Powerful and theoretically well supported.
- The only drawback is the data dependency across the layers in the backward direction.



Why is this a drawback?

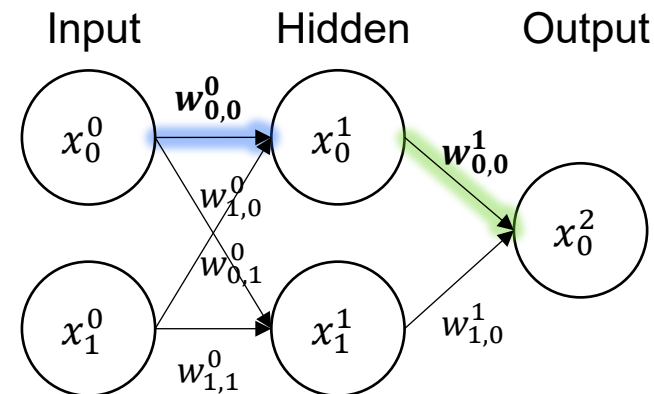
병렬화의 효과성이 떨어짐, 즉 한 레이어에 대한 작업에 대해서만 병렬화 가능 = 막대한 단일 메모리 크기 필요

Backpropagation Finds Optimal Solution

- Backpropagation can be considered as dynamic programming applied to MLP. 항상 후행 레이어는 선행 레이어보다 최적 상태
 - The gradient at each layer is composed of the gradients at the next layer (backward direction).
- By the principle of optimality, the optimal solution of a problem (current layer) exists only based on the optimal solution of the sub-problem (next layer).

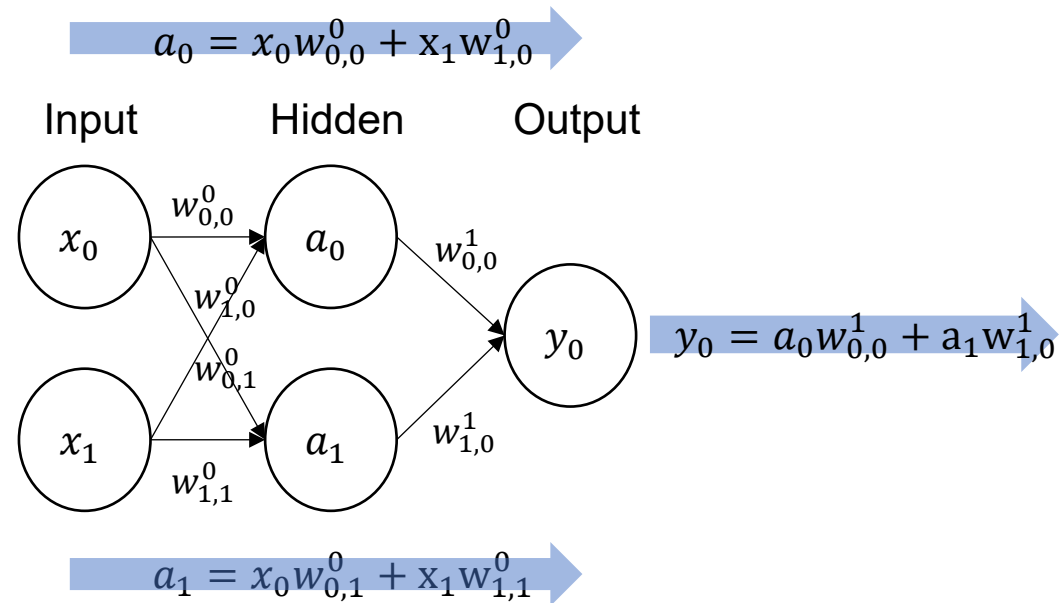
따라서 후행 레이어를 참고하여 선행 레이어를
수정할 시 항상 최적의 방향으로 모델이 개선됨

$w_{0,0}^0$ is optimal only when $w_{0,0}^1$ is optimal!

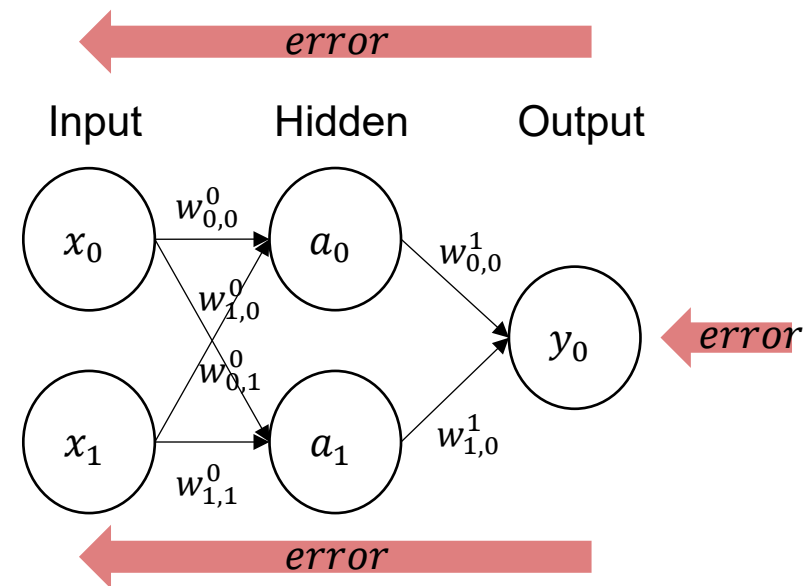


General Terms

- Feed Forward
- Forward Propagation
- Inference



- Backpropagation
- Gradient Calculation



같은 레이어간의 퍼셉트론간에는 정보의 교환이 없음, 따라서 직접적인 연관은 없으나 동일한 입력이 들어가므로 간접적인 연관은 있을 수 있음

Backprop-based Gradient Descent

- Gradient Descent just runs feed-forward and backpropagation repeatedly! 레이어 단위로 데이터 의존성을 지켜서 진행하는 것 이외에는 일반적인 GD와 동일

Algorithm. Gradient Descent

1. **while** termination condition is not met:
2. Calculate the activations going through all the layers.
3. Calculate the gradient of $L(w)$ with respect to the model parameters w .
4. Update the model parameters using the gradients.

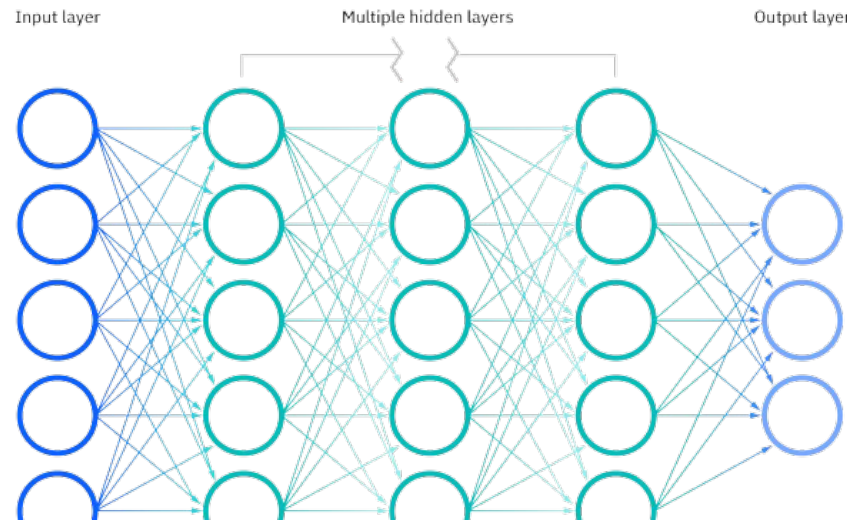
Feed-forward

Backprop.

Update

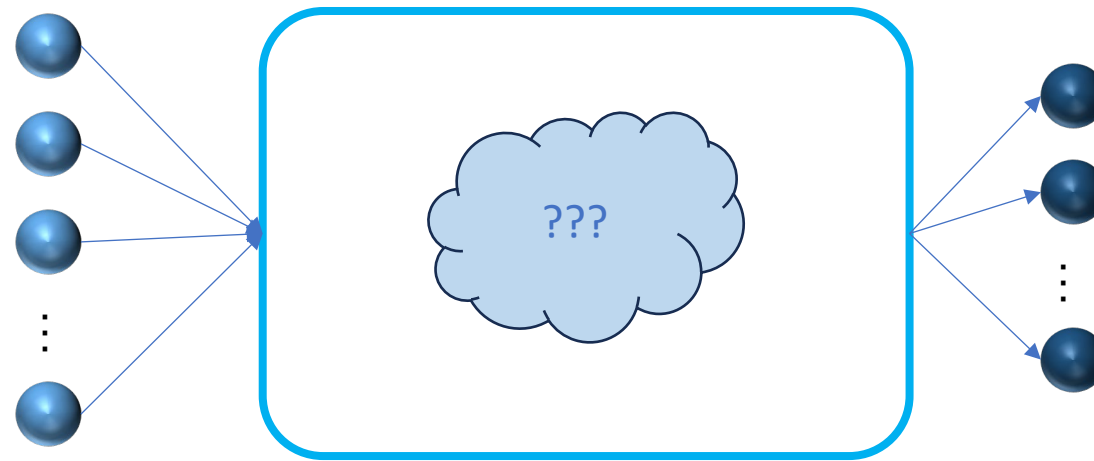
Why Does MLP Work So Well?

- MLP can solve ***non-linear*** problems like SVM with kernel.
- Algorithm is relatively ***simple*** (remind SVM with kernel).
- User-tunable hyper-parameters enable to ***customize*** the model to specific problems.



Versatility

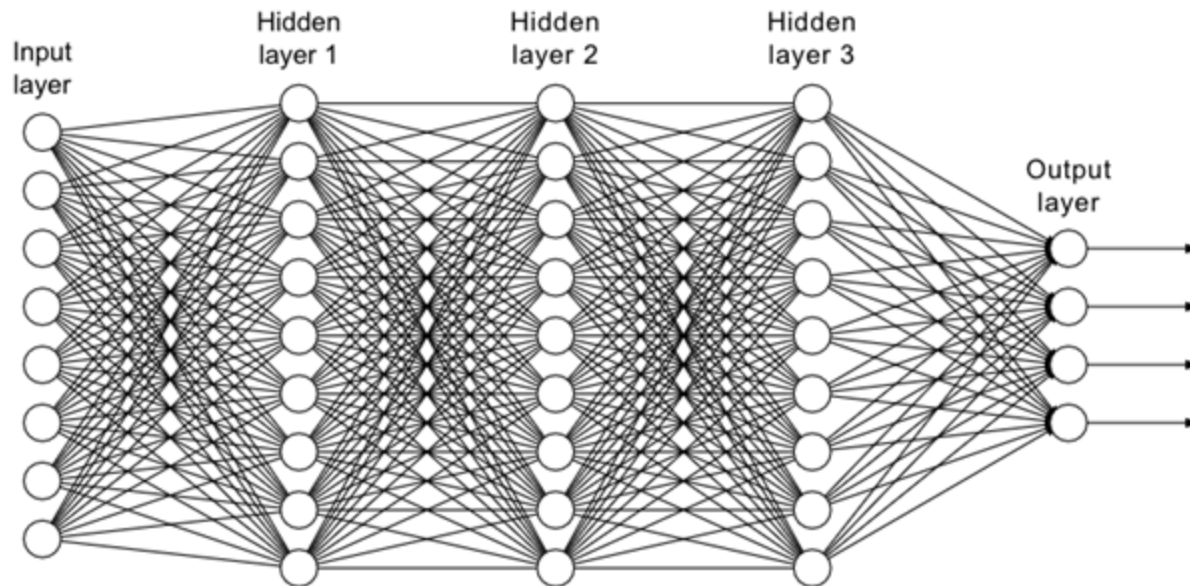
- Input shape: any vectors $\in \mathbb{R}^m$
- Output shape: any vectors $\in \mathbb{R}^n$
- Basically, MLP can fit to any kinds of non-linear mappings.
- The accuracy depends on what it looks like inside the box!



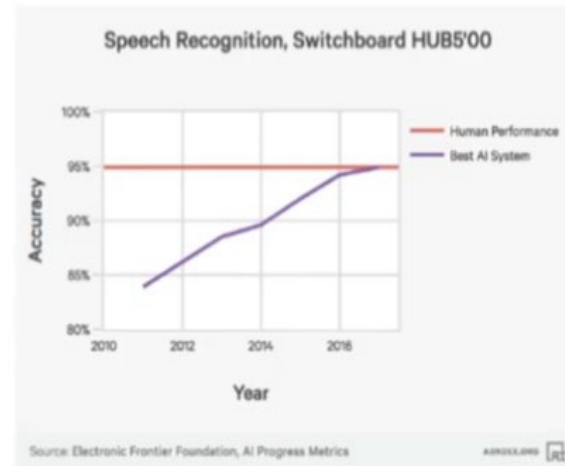
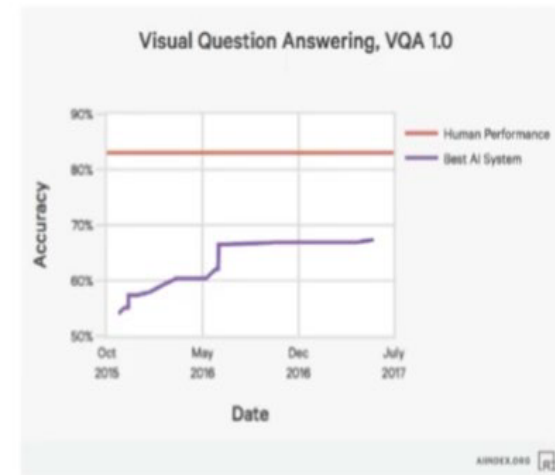
- A Brief History
- Perceptron
- Multi-Layer Perceptron (MLP)
- ***Quick Introduction to Deep Neural Networks***

Artificial Neural Network

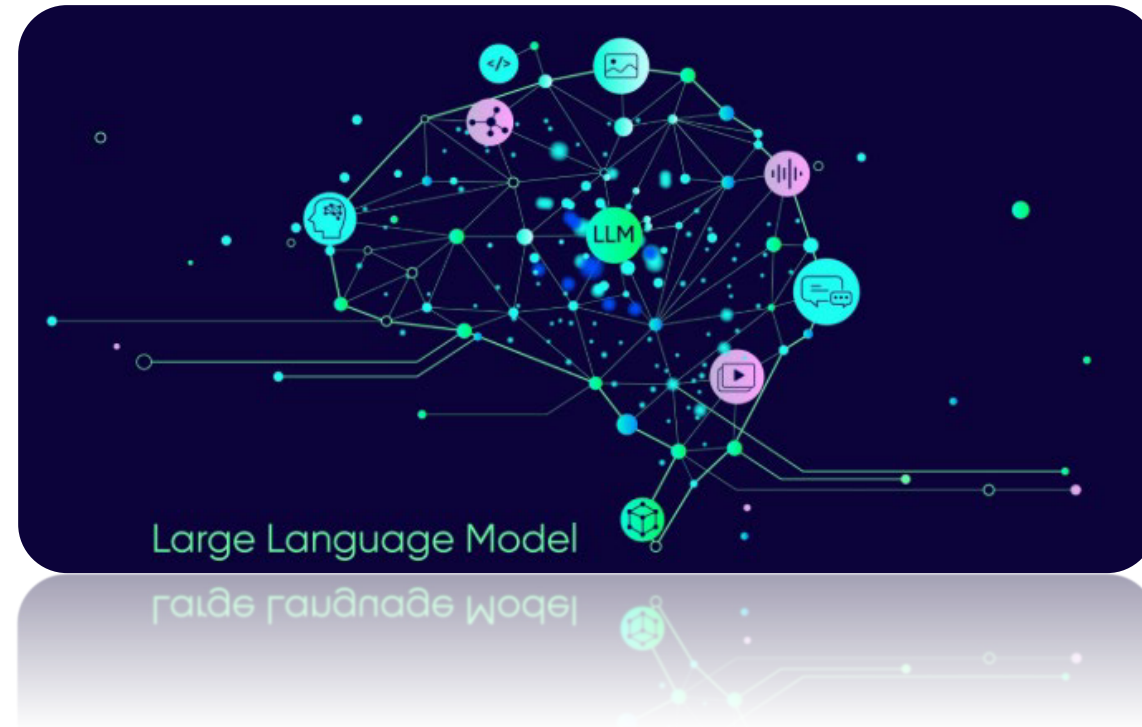
- MLP with a large number of layers.
- Technically, MLP is the same as ANN, however, ANN usually refers to a larger and deeper networks with a variety of connection patterns.



Are They... Better than Human?



Deep learning has evolved toward LLMs.



Here, 'large' indicates at least billions!

Why is deep good?

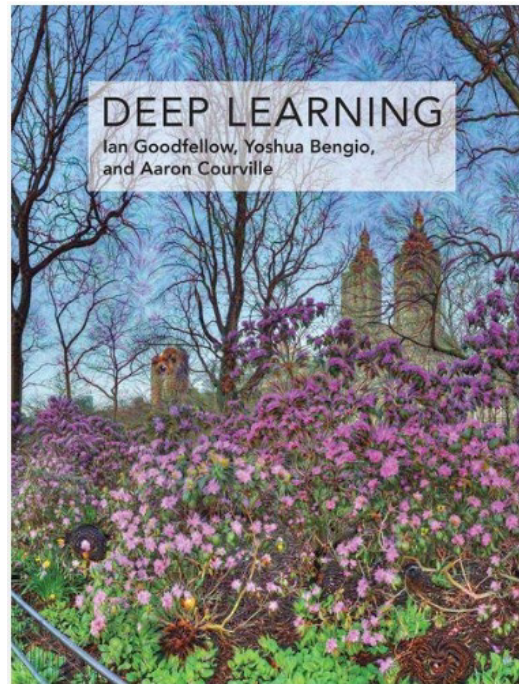
- Hierarchical feature learning
 - The stacked layers learn the inherent hierarchical structures of data.
- Efficiency
 - Smaller layers allow efficient matrix operations (imagine billions of parameters as a single layer).
- Parameter utilization
 - In the over-parameterization regime, the learning capability of many small layers may be better utilized than a single fat layer.
- No manual feature engineering
 - Networks can automatically extract features going through multiple layers!

Several Interesting Topics

- What does your model look like? (CNN, RNN, Transformers, etc.)
- How do you train your models? (SGD, AdaGrad, ADAM, Muon, etc.)
- How do you utilize your system resources? (GPUs, CUDA, CPUs)
- How do you tune hyper-parameters in deep learning?
- How do you make your neural networks stronger? (In terms of accuracy, generalization, scalability, and applicability)

Deep Learning: CSE4317

- A new course offered next semester.
- Deep Learning algorithms, applications, and analysis.



Any Questions?

MLP is just the beginning of Deep Learning.
There are much more fun!

